

Lecture: Prometheus Architecture and Core Components

1 Introduction

Prometheus is an open-source monitoring and alerting system designed for reliability, scalability, and simplicity. It is widely used in cloud-native environments, especially in Kubernetes-based deployments, where services are dynamic and distributed. Unlike traditional monitoring systems, Prometheus follows a pull-based model, stores time-series data locally, and provides a powerful query language called PromQL for analysis and alerting.

2 Core Architecture of Prometheus

Prometheus is composed of several tightly integrated components that work together to collect, store, and query metrics. At the center of the system is the Prometheus Server, which is responsible for scraping metrics, storing them, and serving queries.

The key components include:

- Prometheus Server
- Time-Series Database (TSDB)
- HTTP Server Interface
- Exporters
- Service Discovery

The workflow is as follows: targets expose metrics via HTTP endpoints, Prometheus periodically scrapes these endpoints, stores the data in its internal database, and allows querying via PromQL.

3 Time-Series Data Storage (TSDB)

Prometheus stores all data as time-series, where each data point is identified by:

- Metric name
- Labels (key-value pairs)
- Timestamp
- Value

A single time-series can be represented as:

```
http_requests_total{method="GET", status="200"}
```

The internal storage engine is optimized for append-only workloads. Data is written sequentially, making ingestion efficient. Prometheus stores data locally on disk and organizes it into blocks. Each block contains compressed time-series data over a fixed time window. Recent data is kept in memory for fast access, while older data is compacted and stored on disk. This design ensures high performance for both real-time and historical queries.

4 HTTP Server and Data Retrieval

Prometheus exposes an HTTP server that serves multiple purposes. First, it provides a web-based user interface where users can run queries, visualize metrics, and debug issues. Second, it exposes an API that allows external systems such as Grafana to query metrics. This architecture allows Prometheus to act as both a data collector and a data provider.

5 Pull-Based Data Collection Model

Prometheus uses a pull-based model for collecting metrics, which is fundamentally different from push-based systems. In this model, Prometheus periodically sends HTTP requests to configured targets to retrieve metrics. Each target exposes a `/metrics` endpoint. Prometheus scrapes this endpoint at regular intervals defined in its configuration. This helps to reduce network load, as opposed to push based tools such as new relic.

5.1 How Pull Works

- Application exposes metrics at `/metrics`
- Prometheus sends HTTP GET requests on specified intervals
- Metrics are returned in a text format
- Prometheus parses and stores the data

Example of exposed metrics:

```
# HELP request_count Total number of requests
# TYPE request_count counter
request_count_total 42
```

5.2 Why Pull Model is Important

The pull model provides several advantages. Prometheus has full control over scraping intervals and can detect whether a target is up or down. It simplifies debugging because metrics can be inspected directly at the source. In dynamic environments such as Kubernetes, this model integrates well with service discovery mechanisms.

6 Types of Metrics in Prometheus

Prometheus defines three primary types of metrics. These types determine how data behaves over time and how it should be interpreted.

6.1 Counter

A counter is a cumulative metric that only increases over time. It is typically used to count events such as requests.

```
request_count_total
```

Counters reset only when the application restarts.

6.2 Gauge

A gauge represents a value that can increase or decrease. It is used for measurements such as temperature, memory usage, or CPU utilization.

```
memory_usage_bytes
```

6.3 Histogram

A histogram samples observations and groups them into buckets.

It produces three types of metrics:

- `_bucket`
- `_sum`
- `_count`

Example:

```
request_latency_seconds_bucket  
request_latency_seconds_sum  
request_latency_seconds_count
```

Histograms are useful for latency analysis and percentile calculations.

7 Prometheus Configuration (YAML)

Prometheus is configured using a YAML file, typically named `prometheus.yml`. This file defines how Prometheus discovers and scrapes targets.

7.1 Basic Structure

```
global:  
  scrape_interval: 15s  
  
scrape_configs:  
  - job_name: 'my_app'  
    static_configs:  
      - targets: ['localhost:8000']
```

The `global` section defines default parameters such as scraping frequency. The `scrape_configs` section defines jobs. Each job represents a group of targets. The `targets` field specifies endpoints that Prometheus will scrape.

8 PromQL: Query Language

PromQL (Prometheus Query Language) is used to query and analyze time-series data. It allows users to perform filtering, aggregation, and mathematical operations on metrics.

8.1 Basic Query

```
up
```

This returns whether targets are up (1) or down (0).

8.2 Filtering with Labels

```
http_requests_total{method="GET"}
```

Selects the total, cumulative count of all HTTP requests that used the GET method, as recorded by a Prometheus client libra

9 Putting Everything Together

In a real MLOps pipeline, Prometheus continuously monitors model performance metrics such as request count, latency, and error rates. The workflow becomes: Application exposes metrics → Prometheus scrapes metrics → Data stored in TSDB → Queries via PromQL → Visualization in Grafana → Alerts triggered if thresholds are exceeded. This forms the foundation of systems monitoring.

10 Exporter and Client Library

Usually any microservice will have a metric endpoint exposed for Prometheus. If for some reason it does not, you have to use an exporter, that will take metrics from your service and expose those in its own metric endpoint for Prometheus.

Client library is basically allowing you to use language-specific software package to instrument your application code. Basically, if you are coding in python for example, use python's Prometheus client library to log, define Prometheus metrics.